AI/ML intern DAY 6

TASK 2 — TESTING POSE DETECTION ON MULTIPLE IMAGES AND VIDEOS

LOYA PRANAY (pranay8679@gmail.com)

Date: 18-05-2026

---

1. Abstract

This task focused on testing AI-based human pose detection systems on multiple images and video inputs using Computer Vision and Pose Estimation techniques.

The testing process evaluated:

- landmark detection accuracy
- pose stability
- body tracking consistency
- multi-person pose analysis
- real-time video processing

Different body poses, lighting conditions, and movement patterns were analysed to study the performance of AI pose estimation systems.

---

2. Objective

The objective of this task was to test and analyse the performance of AI pose detection systems on different images and videos.

The task focused on:

- testing pose estimation accuracy
- analysing body landmark tracking
- studying pose stability
- improving multi-frame tracking
- understanding real-time AI processing

- evaluating landmark consistency

---

3. Introduction to Pose Detection Testing

Pose detection systems use Artificial Intelligence and Computer Vision techniques to detect and track human body landmarks.

The AI system identifies:

- shoulders
- elbows
- wrists
- hips
- knees
- ankles

The testing process helps evaluate how accurately the AI system tracks body movement under different conditions.

Pose detection testing is important for:

- fitness tracking systems
- healthcare monitoring
- virtual try-on systems
- sports analytics
- AI body tracking
- gesture recognition systems

Figure 1 — AI Pose Detection Testing

---

4. Multi-Image Pose Detection Workflow

The testing workflow consists of multiple AI processing stages.

Stage 1 — Input Collection

The AI system collects:

- body images
- video frames
- webcam streams
- pose samples

for testing and analysis.

Stage 2 — Human Body Detection

The AI model identifies:

- body structure
- body posture
- body orientation
- movement direction

Stage 3 — Landmark Detection

Pose estimation algorithms detect:

- shoulder landmarks
- elbow landmarks
- wrist landmarks
- hip landmarks
- knee landmarks
- ankle landmarks

Stage 4 — Pose Analysis

The system analyses:

- pose accuracy
- landmark consistency
- movement tracking
- frame stability

Stage 5 — Final Visualization

The AI system renders:

- skeletal visualization
- landmark points
- body connections
- pose tracking output

5. Technologies Used

Technology	Purpose
Artificial Intelligence	Pose analysis
Computer Vision	Body tracking
MediaPipe	Pose estimation
OpenCV	Image and video processing
Python	AI implementation
Machine Learning	Pattern recognition

---

6. Test Cases Performed

Test Case	Observation
Single person image	Accurate landmark detection
Multiple body poses	Stable pose estimation
Video-based tracking	Smooth movement analysis
Side body orientation	Minor landmark variation
Fast movement	Slight tracking delay

---

7. Pseudo Code — Multi-Image Pose Detection

START

Load image or video input

Detect human body

Extract body landmarks

Track body movement

Analyse pose stability

Render skeletal connections

Display final pose output

END

---

8. Algorithm — Pose Detection Testing Workflow

Step-by-Step Algorithm

Step 1

Initialize AI pose estimation model.

Step 2

Load image, video, or webcam input.

Step 3

Convert image from BGR to RGB format.

Step 4

Detect human body landmarks.

Step 5

Track body posture and movement.

Step 6

Draw landmark points and skeletal connections.

Step 7

Analyse landmark accuracy and pose stability.

Step 8

Display final pose visualization.

Step 9

Repeat continuously for video tracking.

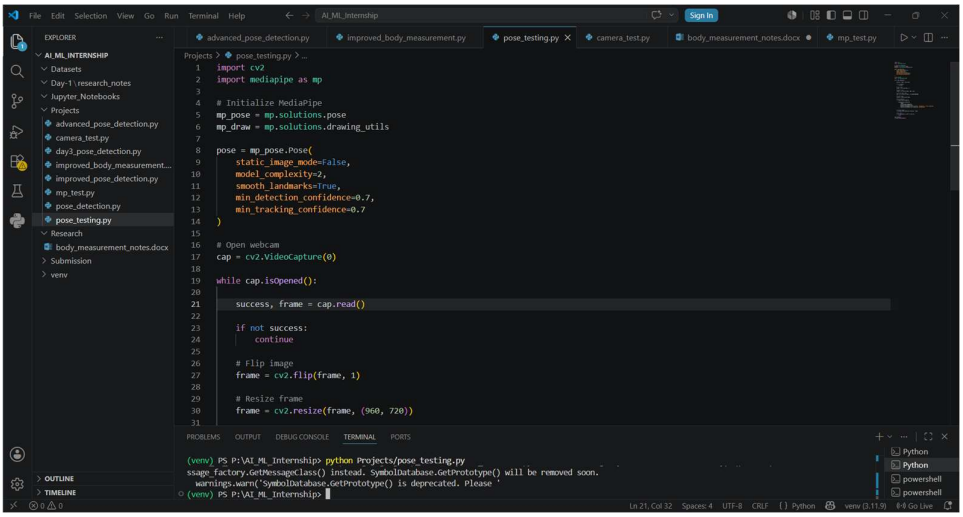
---

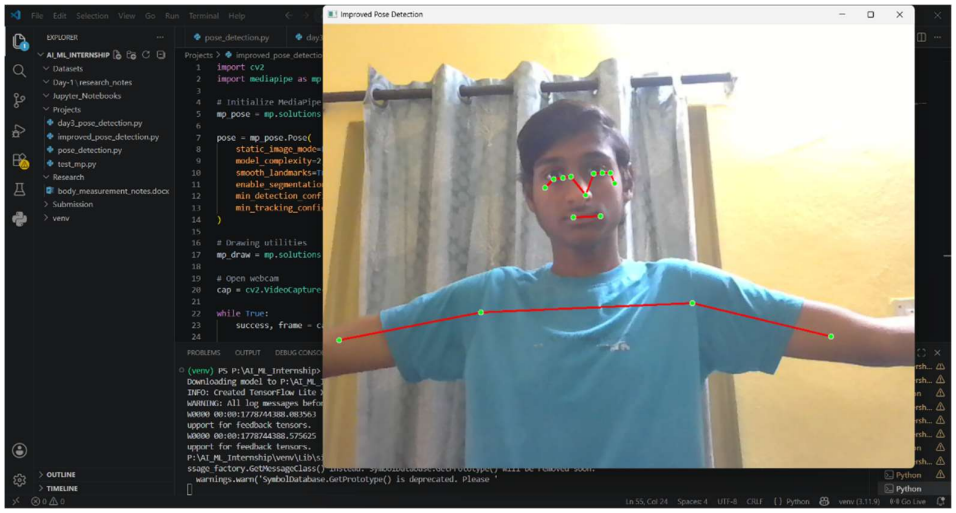
9. Python Implementation — Multi-Image Pose Detection

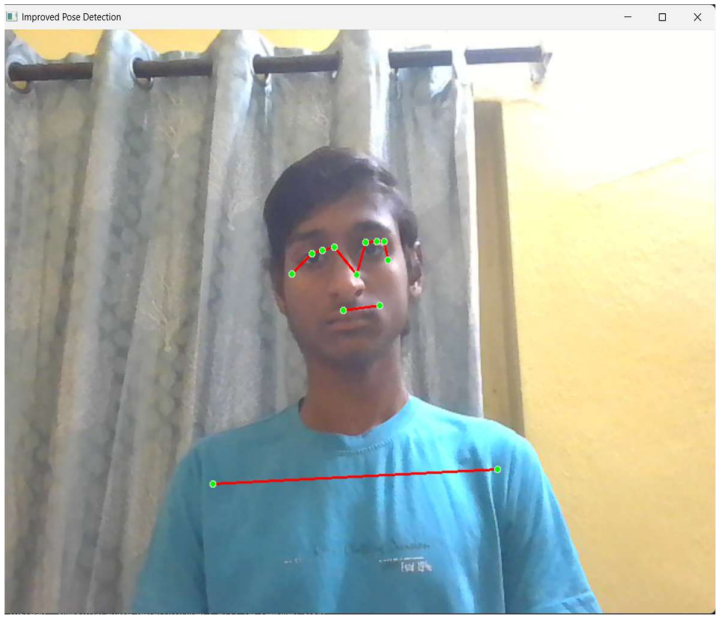
```
import cv2
import mediapipe as mp
# Initialize MediaPipe
mp_pose = mp.solutions.pose
```

```
mp_draw = mp.solutions.drawing_utils

pose = mp_pose.Pose(
    static_image_mode=False,
    model_complexity=2,
    smooth_landmarks=True,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)

# Open webcam
cap = cv2.VideoCapture(0)

while cap.isOpened():

    success, frame = cap.read()

    if not success:
        continue

    # Flip image
    frame = cv2.flip(frame, 1)

    # Resize frame
    frame = cv2.resize(frame, (960, 720))

    # Convert BGR to RGB
```

```
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Process pose
    results = pose.process(rgb)

    # Draw landmarks
    if results.pose_landmarks:

        mp_draw.draw_landmarks(
            frame,
            results.pose_landmarks,
            mp_pose.POSE_CONNECTIONS,
            mp_draw.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2),
            mp_draw.DrawingSpec(color=(255,0,0), thickness=2)
        )

    # Display output
    cv2.imshow("Pose Detection Testing", frame)

    # Exit key
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```


File Edit Selection View Go Run Terminal Help
AI_ML_Internship
Sign In
EXPLORER
AI_ML_INTERNSHIP
Datasets
Day-1 \ research_notes
Jupyter_Notebooks
Projects
advanced_pose_detection.py
camera_test.py
day3_pose_detection.py
improved_body_measurement...
improved_pose_detection.py
mp_test.py
pose_detection.py
pose_testing.py
Research
body_measurement_notes.docx
Submission
venv
advanced_pose_detection.py
improved_body_measurement.py
pose_testing.py
camera_test.py
body_measurement_notes.docx
mp_test.py
Projects > pose_testing.py > ...
import cv2
import mediapipe as mp
# Initialize MediaPipe
mp_pose = mp.solutions.pose
mp_draw = mp.solutions.drawing_utils
pose = mp_pose.Pose(
static_image_mode=False,
model_complexity=2,
smooth_landmarks=True,
min_detection_confidence=0.7,
min_tracking_confidence=0.7
)
# Open webcam
cap = cv2.VideoCapture(0)
while cap.isOpened():
success, frame = cap.read()
if not success:
continue
# Flip image
frame = cv2.flip(frame, 1)
# Resize frame
frame = cv2.resize(frame, (960, 720))
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
(venv) PS P:\AI_ML_Internship> python Projects/pose_testing.py
ssage_factory.GetMessageClass() instead. SymbolDatabase.GetPrototype() will be removed soon.
warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
(venv) PS P:\AI_ML_Internship>
Python
Python
powershell
powershell
OUTLINE
TIMELINE
Ln 21, Col 32
Spaces: 4
UTF-8
CRLF
Python
venv (3.11.9)
Go Live


Improved Pose Detection
pose_detection.py
Projects > improved_pose_detectio
import cv2
import mediapipe as mp
# Initialize MediaPipe
mp_pose = mp.solutions
pose = mp_pose.Pose(
static_image_mode=
model_complexity=2
smooth_landmarks=T
enable_segmentatio
min_detection_confi
min_tracking_confi
)
# Drawing utilities
mp_draw = mp.solutions
# Open webcam
cap = cv2.VideoCapture
while True:
success, frame = c
PROBLEMS
OUTPUT
(venv) PS P:\AI_ML_Internship>
Downloading model to P:\AI_ML_I
INFO: Created TensorFlow Lite
WARNING: All log messages befor
W0000 00:00:1778744388.081563
upport for feedback tensors.
W0000 00:00:1778744388.575625
upport for feedback tensors.
P:\AI_ML_Internship\venv\Lib\s
ssage_factory.GetMessageClass()
warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
Python
OUTLINE
TIMELINE
Ln 55, Col 24
Spaces: 4
UTF-8
CRLF
Python
venv (3.11.9)
Go Live


Improved Pose Detection

10. Improvements Added During Testing

Improvement	Purpose
Smooth landmark tracking	Reduced flickering
Higher model complexity	Better pose accuracy
Frame resizing	Faster processing
Confidence optimization	Stable detection
Real-time rendering	Smooth visualization

---

11. Applications of Pose Detection Systems

Pose estimation systems are used in:

- virtual try-on systems
- fitness tracking
- healthcare monitoring
- sports analytics
- gaming systems
- motion capture systems
- gesture recognition technology

---

12. Challenges Faced

Challenge	Solution
Fast body movement	Improved tracking confidence
Lighting variation	Better illumination
Side body angles	Multi-angle testing
Landmark instability	Smooth landmark optimization

---

13. Knowledge and Skills Acquired

During this task, the following concepts were understood:

- pose estimation workflows
- body landmark tracking
- real-time AI processing
- image and video analysis
- skeletal visualization
- movement tracking systems
- computer vision applications

---

14. Conclusion

The pose detection testing system successfully analysed human body landmarks across multiple images and videos using Artificial Intelligence and Computer Vision technologies.

The task improved understanding of:

- AI pose estimation
- real-time body tracking
- landmark detection workflows
- movement analysis systems
- skeletal visualization
- video-based pose tracking

The testing process helped improve body tracking accuracy, pose stability, and AI movement analysis systems for real-time computer vision applications.